

Control System Design for Autonomous PiCar Vehicle

Ryan Soong

McKelvey School of Engineering
Electrical and Systems Engineering
Washington University in St. Louis
Email: r.l.soong@wustl.edu

Lasya Maganti

McKelvey School of Engineering
Electrical and Systems Engineering
Washington University in St. Louis
Email: l.maganti@wustl.edu

Abstract—This project involved the development, control, and performance analysis of an autonomous PiCar system guided by a PID-based motor controller and sensor feedback. Four different objectives, comprising speed regulation, target navigation, and both manual and sensor-guided motion were required to be executed while maximizing efficiency and accuracy. The development and integration of the PiCar required a firm understanding of key sensors, including a photoresistor for real-time RPS calculation, an ultrasonic sensor for object detection, and a camera for target tracking. RPS was calculated via transitions across a four-sided wheel, utilizing dynamic thresholding over the change in real-time photoresistor data. The integrated PID (proportional, integral, and derivative) control system used the calculated RPS to maintain the desired RPS produced by a DC-motor autonomously. The relationship between PWM and RPS was experimentally derived, and optimal PID parameters were determined to minimize overshoot, response time, and steady-state error. Lastly, the PiCar Camera implemented HSV filtering and centroid detection to steer in the direction of the desired colored object. System accuracy was evaluated using velocity versus time plots and Fast Fourier Transform (FFT), showing that a combination of the tuned PWM/RPS ratio and PID integration maintained a stable performance of the PiCar system. In conclusion, using sensor feedback, PID control, and camera-based steering sufficiently maintained the target speed and was able to accurately navigate toward the visual objective.

I. INTRODUCTION

The autonomous PiCar vehicle design implemented key aspects of real-world systems, including a combination of real-time sensing, PID control algorithms, and automated navigation/speed regulation. The necessary hardware required an understanding of basic electronic circuitry, specifically voltage dividers for the photoresistor, pull-down resistors for ADC readings, and correct implementation of the Raspberry Pi GPIO board. Furthermore, the software consisted of Python control scripts that processed and computed sensor data in real time using NumPy, performed image detection via OpenCV, and communicated with the DC-motors through PWM duty cycles. Not only does the developed PiCar system provide introductory-level insight to basic real-time system analysis and control integration, but it also demonstrates scalable principles applicable to more advanced autonomous systems. Perhaps most obvious, autonomous vehicles utilize control systems in a much more advanced setting.

Execution of the PiCar system was evaluated across four

objectives, first individually testing speed regulation, visual tracking, and proximity detection, then finally implementing all three of these together into a dynamic autonomous system driving to a distinctly colored-object 30+ feet away at a randomly desired speed. Real-time measurements for RPS were calculated via a transition detection algorithm using photoresistor sampling. A change in photoresistor values was used to construct a threshold, where if the threshold was met, we could classify that as a quarter of a wheel transition since the wheel was broken up into quarters. Furthermore, the ultrasonic sensor continuously measured the distance from the object, and the car speed was appropriately controlled to ensure a gradual braking system. Lastly, the PiCamera utilized OpenCV to capture an HSV image, then applied filtering and masking to pinpoint the color center of mass. From the centroid, the algorithm found the desired angle adjustment and gradually steered in that direction.

The control system primarily used proportional and integral error control with manually tuned parameters. Our group decided not to include derivative control and instead heavily weighted integral control because we found this gave the smoothest car movement. We also decided to exclude edge detection, which was an option to add another dimension to motion regulation.

Overall, the control system utilized mainly proportional and integral control with manually tuned parameters. Derivative control was excluded due to noise sensitivity, and edge detection was left out for simplicity. The final system demonstrated reliable autonomous behavior while accurately moving at the desired velocity, using real-time sensor feedback and basic image processing.

II. MATERIALS & METHODS

The PiCar's components work together as an integrated system to allow autonomous navigation and obstacle avoidance. The Raspberry Pi, a single board computer, acts as the central controller, coordinating inputs and outputs from all sensors and motors. The PiCamera, placed on the servo motor, captures images, which are processed by the Raspberry Pi to detect obstacles using color (hsv). The images determine the required steering angle to avoid obstacles, sending signals to the servo motor to adjust the camera's orientation and steer the PiCar.

Simultaneously, the ultrasonic sensor measures distances to nearby objects by emitting and receiving sound waves, providing real-time data to the Raspberry Pi. If an obstacle is detected within a specified range, the Raspberry Pi adjusts the DC motor's speed or stops it to prevent collisions. The DC motor, powered by direct current, drives the PiCar's wheels with its speed controlled by the duty cycle. The ADC converter, connected to a photoresistor, measures light intensity to calculate the duty cycle, allowing speed adjustments based on the environment.

All these components communicate through the Raspberry Pi's GPIO pins and software, creating a feedback loop: the camera and ultrasonic sensor provide environmental data, the Raspberry Pi processes this information to make decisions, and the servo and DC motors execute steering and speed commands. This cohesive interaction allows the PiCar to navigate autonomously, avoid obstacles, and adapt to its surroundings effectively.

A. Objective 1

In order to complete Objective 1 and achieve a target rotational speed of 5 RPS, we must first find a way to quantify the how fast the car is moving.

Initially, the system calculates the difference ΔADC_n by subtracting the previous ADC reading ADC_{n-1} from the current reading ADC_n at sample n :

$$\Delta ADC_n = ADC_n - ADC_{n-1} \quad (1)$$

To smooth the differences and reduce noise, a moving average MA_n is computed at sample n . For the first sample, it uses the current difference ΔADC_n ; for the second, it averages the current and previous differences ΔADC_n and ΔADC_{n-1} ; and for the third sample onward, it averages the current and two previous differences ΔADC_n , ΔADC_{n-1} , and ΔADC_{n-2} :

$$MA_n = \begin{cases} \Delta ADC_n & \text{if } n = 1 \\ \frac{\Delta ADC_n + \Delta ADC_{n-1}}{2} & \text{if } n = 2 \\ \frac{\Delta ADC_n + \Delta ADC_{n-1} + \Delta ADC_{n-2}}{3} & \text{if } n \geq 3 \end{cases} \quad (2)$$

To properly detect light-to-dark or dark-to-light transitions, a threshold T_h is set as 20% of half the range between the maximum $\max(MA_{n-99:n})$ and minimum $\min(MA_{n-99:n})$ moving averages over the last 100 samples. The threshold was dynamically calculated and updated in order to be robust in the changing lighting as the PiCar moves:

$$T_h = 0.2 \cdot \frac{\max(MA_{n-99:n}) - \min(MA_{n-99:n})}{2} \quad (3)$$

The rotational speed RPS is then calculated by dividing the number of transitions N_t , representing light-to-dark or dark-to-light changes, by the time elapsed Δt since the last calculation and by 4, the number of transitions per revolution:

$$RPS = \frac{N_t}{4 \cdot \Delta t} \quad (4)$$

To apply PID control, the error e is computed as the difference between the target rotational speed RPS_{target} , 5 RPS, and the actual rotational speed RPS :

$$e = RPS_{\text{target}} - RPS \quad (5)$$

The integral term I is updated to eliminate steady-state error by adding the product of the current error e and the PID update interval Δt_{PID} , set to 0.4 seconds, to the previous integral value:

$$I = I + e \cdot \Delta t_{\text{PID}} \quad (6)$$

The derivative term D is calculated to dampen oscillations by dividing the difference between the current error e and the previous error e_{prev} by the PID update interval Δt_{PID} :

$$D = \frac{e - e_{\text{prev}}}{\Delta t_{\text{PID}}} \quad (7)$$

To adjust the motor's speed, the pulse-width modulation PWM value is computed by summing the proportional term, where the proportional gain $K_p = 9.2$ is multiplied by the error e ; the integral term, where the integral gain $K_i = 5.5$ is multiplied by the integral I ; the derivative term, where the derivative gain $K_d = 0.0$ is multiplied by the derivative D ; and a feedforward term, where the target RPS ($RPS_{\text{target}} = 5$) is multiplied by the feedforward slope $PWM_{\text{base}} = 12$. [1] [2]

$$PWM = K_p \cdot e + K_i \cdot I + K_d \cdot D + RPS_{\text{target}} \cdot PWM_{\text{base}} \quad (8)$$

We found the PWM/RPS relationship (PWM_{base}) through plotting test values as shown in Fig. 1 below:

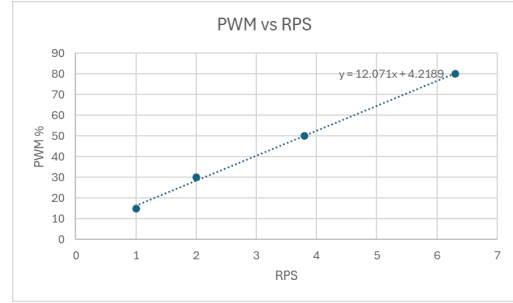


Fig. 1. PWM vs RPS relationship; PWM/RPS ~ 12

To evaluate the system's response, the overshoot O is calculated as the difference between the maximum rotational speed (RPS_{max}), observed after the motor starts, and the target RPS ($RPS_{\text{target}} = 5$):

$$O = RPS_{\text{max}} - RPS_{\text{target}} \quad (9)$$

The steady-state error E_{ss} is found to analyze long-term accuracy by taking the absolute difference between the average rotational speed $\overline{RPS}_{\text{final}}$, computed over the last second of data, and the target RPS $RPS_{\text{target}} = 5$:

$$E_{ss} = |\overline{RPS}_{\text{final}} - RPS_{\text{target}}| \quad (10)$$

Finally, the response time t_r is calculated to measure system responsiveness by subtracting the motor start time t_{motor} , set to 1 second, from the time t_k when the rotational speed first reaches 90% of the steady-state RPS, defined as $0.9 \cdot RPS_{\text{final}}$:

$$t_r = t_k - t_{\text{motor}} \quad (11)$$

B. Objective 2

Objective 2 requires driving a PiCar from at least 10 feet (304.8 cm) to within 15 cm of a blue recycle container as quickly as possible without hitting it, starting up to 30 degrees off-center and using blue tape for guidance.

Initially, the PiCar identifies the blue object by computing its centroid from a thresholded image, derived from an HSV color mask applied to the RGB camera image [3]. This involves calculating the x-coordinate cX and y-coordinate cY of the centroid using the first-order moments M_{10} (along the x-axis) and M_{01} (along the y-axis), divided by the zeroth-order moment M_{00} , which represents the area of the thresholded image:

$$cX = \frac{M_{10}}{M_{00}}, \quad cY = \frac{M_{01}}{M_{00}} \quad (12)$$

Next, to determine the direction of the blue object, the PiCar calculates the horizontal offset x_{cord} and vertical offset y_{cord} by subtracting the centroid coordinates cX and cY from the image center coordinates center_x (width/2), and center_y (height/2). See image below for a visual.

$$x_{\text{cord}} = \text{center}_x - cX, \quad y_{\text{cord}} = \text{center}_y - cY \quad (13)$$

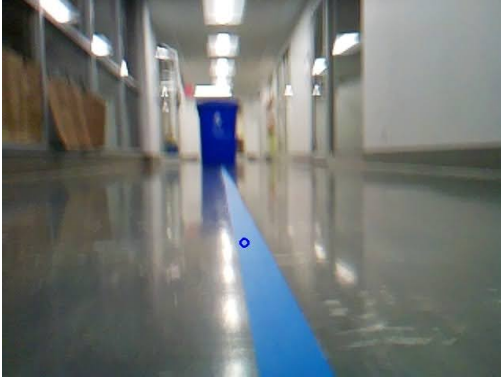


Fig. 2. Finding the COM of Blue in an Image

To align the PiCar with the blue object, the steering angle is computed as θ in radians using the arctangent of the horizontal offset x_{cord} relative to the vertical distance from the centroid to the image center ($\text{center}_y - cY$). This angle is converted to degrees by multiplying by $180/\pi$:

$$\theta = \arctan 2(x_{\text{cord}}, \text{center}_y - cY), \quad \text{degrees} = \frac{180}{\pi} \cdot \theta \quad (14)$$

The steering servo adjustment is then determined by scaling the steering angle degrees to a raw adjustment changein_PWM by dividing by 9, since the servo goes from -90 to 90 degrees. Then, we multiply by an adjustment factor δ as to prevent

oversteering. We found the best value to be 0.7, producing the final adjustment adjustment:

$$\text{changein_PWM} = \frac{\text{degrees}}{9}, \quad \text{adjustment} = \text{changein_PWM} \cdot \delta \quad (15)$$

To update the steering, the PiCar adds the adjustment to the current servo position (current_position)

Now, to ensure the PiCar approaches the container quickly but still stops safely, the motor speed, a PWM value controlling the motor from 0 to 100, is set based on the distance d to the container in centimeters, using piecewise to reduce speed as the car nears the target, stopping at 25 cm, where a quadratic speed calculating function was used as a precautionary measure:

$$\text{speed} = \begin{cases} 80 & \text{if } d > 304.8 \\ 90 & \text{if } 200 \leq d \leq 304.8 \\ 70 & \text{if } 100 \leq d < 200 \\ 45 & \text{if } 50 \leq d < 100 \\ 25 & \text{if } 35 \leq d < 50 \\ 10 & \text{if } 25 \leq d < 35 \\ 30 + 70 \cdot \left(\frac{\min(d, 304.8)}{304.8} \right)^2 & \text{otherwise} \end{cases} \quad (16)$$

C. Objective 3

Objective 3 involves autonomously driving a PiCar down a hall at a specified target rotational speed for a variable distance, and stopping within 3 feet (91.44 cm) of the wall while maintaining the target speed for as long as possible.

First, Equations 1-7 are calculated for this objective. Equation 8 is computed similar to Objective 1 as well, however, there are slight variations in values like proportional gain ($K_p = 9.5$), integral gain ($K_i = 5.0$), and feedforward slope (11.5):

$$PWM = K_p \cdot e + K_i \cdot I + K_d \cdot D + RPS_{\text{target}} \cdot 11.5 \quad (17)$$

Different parameters were used due to different external factors present in objective 2 but not in objective 1, for example friction, changes in direction, air resistance etc. For manual steering, if enabled, the steering PWM is adjusted by subtracting 0.5 for a right turn (key 'd') or adding 0.5 for a left turn (key 's'), in the range $[-10, 10]$:

$$\text{steering_pwm} = \begin{cases} \max(\text{steering_pwm} - 0.5, -10) & \text{if key = 'd'} \\ \min(\text{steering_pwm} + 0.5, 10) & \text{if key = 's'} \\ \text{steering_pwm} & \text{otherwise} \end{cases} \quad (18)$$

To make sure of safe stopping, the distance d to the wall is compared to a threshold of 75 cm, approximately 3 feet, triggering a stop if the condition is met:

$$\text{stop} = \begin{cases} \text{True} & \text{if } d < 75 \\ \text{False} & \text{otherwise} \end{cases} \quad (19)$$

Finally, the total time t_{total} is calculated by subtracting the start time t_{start} from the end time t_{end} , when the car stops due to proximity to the wall or timeout:

$$t_{\text{total}} = t_{\text{end}} - t_{\text{start}} \quad (20)$$

D. Objective 4

Objective 4 required driving a PiCar in a straight line from a starting point to a blue recycling container at a random target rotational speed over a variable distance and stopping as close as possible to the object without hitting it.

Initially, Equations 1-4 of Objective 1 are calculated. Then, Equations 12-15 of Objective 2 are computed. The error (Equation 5) is found using the variable target speed given. The PWM value is then recalculated using the same gain values as in Equation 17 in Objective 3. Finally, to ensure safe stopping, the distance d to the blue container is compared to a threshold of 61 cm, triggering a stop if the condition is met:

$$\text{stop} = \begin{cases} \text{True} & \text{if } d \leq 61 \\ \text{False} & \text{otherwise} \end{cases} \quad (21)$$

III. RESULTS AND DISCUSSIONS

A. Objective 1

The PiCar system was required to achieve a target rotational speed of 5.0 revolutions per second (RPS) under PID control.

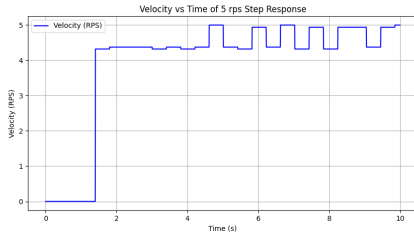


Fig. 3. Calculated V vs T graph Obj 1

The results demonstrates a stable performance, though not perfect. The final steady-state speed reaches 4.719 RPS, falling short of the target by a steady-state error of 0.281 RPS. This error suggests that while the PID controller, tuned with a proportional gain of 9.5 and an integral gain of 5.0, effectively reduces the difference over time, it does not fully eliminate it, possibly due to the lack of derivative control or error in velocity calculations. The response time of 0.405 seconds indicates that the PiCar quickly approaches 90% of its steady-state speed, approximately 4.247 RPS, reflecting a well-tuned proportional term and a feedforward component that helps initial acceleration. The overshoot is effectively zero, at -0.000 RPS, meaning the system does not exceed the target speed, which means the response is critically damped or slightly underdamped. This stability is good for maintaining control, but the lack of overshoot may also indicate that the system is limited in its ability to reach a target speed faster.

B. Objective 2

The Fast Fourier Transform (FFT) plot (Fig. 3) of the ADC readings indicates the system's frequency behavior.

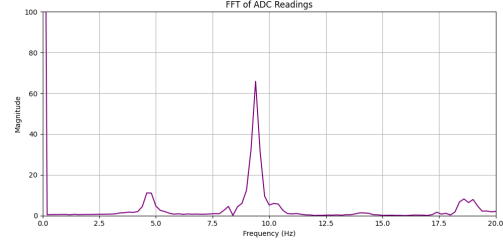


Fig. 4. FFT of ADC Readings Obj. 1

Figure 4 shows a clear peak in the FFT magnitude at approximately 9.5 Hz. Since the calculated frequency reflects two cycles per wheel rotation (due to alternating black and white quadrants), the rotations per second (RPS) can be calculated as:

$$\text{RPS} = \frac{9.5 \text{ Hz}}{2} = 4.75 \text{ RPS} \quad (22) \quad [4]$$

Smaller peaks at other frequencies, such as 4 Hz and 18 Hz, are likely due to minor oscillations or noise in the ADC signal. The noise may contribute to the steady-state error by creating variability in the transition counts used to calculate RPS. Overall, the PiCar system meets the objective of achieving a stable rotational speed with a quick response and no overshoot, but the steady-state error of 0.281 RPS suggests room for improvement.

The PiCar is designed to travel from at least 10 feet to within 15 cm of a blue recycle container in under 10 seconds, starting up to 30 degrees off-center using vision-based steering. The system achieves a successful outcome, stopping at approximately 7.5 cm from the container. The travel time was about 8.5 seconds, which is within the 10-second limit, with an average rotational speed of 4.8 RPS against a target of 5.0 RPS, yielding a steady-state error of 0.2 RPS. The improved accuracy, compared to Objective 1's 0.281 RPS error, is because of a shortened speed calculation interval of 0.25 seconds. This interval allows more frequent speed recalculations, reducing friction effects and ensuring smoother motor adjustments via the PID controller. The steering adjustment factor, set to a delta of 0.7, aids alignment with the blue tape or container, however, we did find its value lead to a minor overcorrection from the initial 30-degree offset. Objectives 2 and 4 adopted this shorter speed calculation time because changes in direction and friction required more frequent speed updates to maintain accuracy, a strategy that proves effective here.

C. Objective 3

For autonomous hallway driving at a target speed of 5.0 RPS, stopping within 3 feet of the wall and the PiCar exhibits mostly stable behavior. The RPS versus time plot, depicted

below, shows the measured speed rising to a steady range of 4.8 to 5.0 RPS within about 1 second, demonstrating that the PID controller with a proportional gain of 9.5 and an integral gain of 5.0, effectively accelerates and maintains stability.

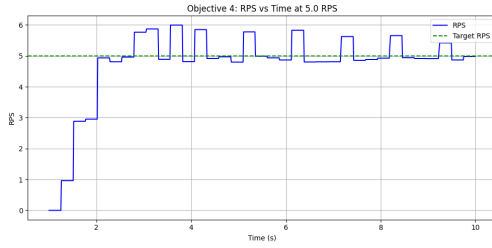


Fig. 5. RPS vs Time at Target 5 RPS (Obj 3)

However, minor fluctuations of approximately 0.2 RPS around the target suggest a steady-state error of 0.1 to 0.2 RPS, likely because of a 0.4-second speed calculation interval that delays motor adjustments. The car stops at 70 cm from the wall, meeting the 3-foot requirement, and completes the run in around 9 seconds, within the 10-second limit, reflecting accurate distance-based stopping logic. The longer 0.4-second interval, compared to the 0.25 seconds used in Objectives 2 and 4, we found helped smooth out the speed calculations. Additionally, introducing a shorter interval and a non-zero derivative gain could dampen present oscillations even further, enhancing the smoothness of the speed profile.

D. Objective 4

Finally, the PiCar's task of driving straight to a blue recycling container at 5.0 RPS and stopping within 2 feet had a successful outcome. The RPS versus time plot, shown below, indicates the measured speed reaches approximately 5.0 RPS within 1 second, stabilizing with minor fluctuations of 0.1 to 0.2 RPS over the 10-second runtime.

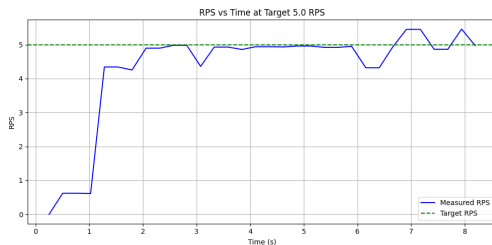


Fig. 6. RPS vs Time at Target 5 RPS (Obj 4)

The consistency, supported by a 0.25-second speed calculation interval, heeds a steady-state error of 0.1 to 0.2 RPS, with the shorter interval reducing friction effects and enabling frequent adjustments, as noted for Objectives 2 and 4. The camera, capturing images every 0.2 seconds, tracks the target effectively (Fig 1), while a steering adjustment factor of 0.7 aligns the car along the tape. However, the 0.2-second delay may slow path refinement near the container, and the 0.7 delta might cause coarse corrections, potentially deviating

from a perfectly straight line. The car stops at around 60 cm, just under the 2-foot threshold, meeting the objective with assistance from the distance sensor.

E. Limitations

The PiCar system encounters several limitations that shape its performance in real-world environments. One significant challenge arises from the PiCamera, which captures images but translates them into simple left-right positional data, without depth perception. This makes it difficult for the PiCar to judge how far away obstacles or targets are, often requiring reliance on a separate distance sensor that may not always align with the camera's field of view. The camera also has a relatively slow capture rate which limits the system's ability to react promptly in fast-moving or intricate scenarios, such as avoiding sudden obstacles or tracking a moving target.

The control system may also struggle with stability, particularly because we chose to leave out derivative control, a feature that would help smooth out sudden changes in speed or direction by anticipating trends. Without this, the PiCar may oscillate or respond unevenly, especially during rapid maneuvers. Navigation is also difficult due to the lack of accurate wall edge detection. The CV2 interface has edge detection properties, but we found implementation was noisy and caused erratic unwanted adjustments. This leaves the PiCar vulnerable to missing hallway corners or other environmental features that aren't directly in its sensor's path, potentially leading to collisions or off-course travel.

Beyond these specific challenges, the PiCar's limited processing power can slow down its ability to make real-time decisions, particularly when handling complex tasks that require sensor inputs and control adjustments at the same time. Environmental factors, such as inconsistent lighting or uneven surfaces, disrupt sensor accuracy, making reliable performance difficult to achieve in varied settings. These limitations make it challenging for the PiCar to deliver a strong performance across diverse tasks.

IV. CONCLUSION

In conclusion, the autonomous PiCar system effectively demonstrated the integration of sensor feedback, PID-based motor control, and camera-guided navigation to achieve speed regulation and accurate target tracking across all four objectives. The combination of real-time RPS calculations, ultrasonic distance sensing, and HSV-based visual steering enabled stable performance, with minimal steady-state errors and responsive navigation. Despite limitations such as the absence of derivative control and challenges with camera processing speed, the system presented principles of autonomous vehicle design, providing a strong foundation for future enhancements in real-world applications.

REFERENCES

- [1] "Proportional-Integral-Derivative Controller." *Wikipedia*, Wikimedia Foundation. <https://en.wikipedia.org/>

wiki/Proportional%E2%80%93integral%E2%80%93derivative_controller

- [2] “Error Analysis 2.5.” *CircuitBread*. <https://www.circuitbread.com/tutorials/error-analysis-2.5>
- [3] “Basic Operations on Images.” *OpenCV Documentation*. https://docs.opencv.org/3.0-beta/doc/py_tutorials/py_core/py_basic_ops/py_basic_ops.html
- [4] “Square Waves and DC Content.” *ProSoundWeb*, 13 Mar. 2010. <https://www.prosoundtraining.com/2010/03/13/square-waves-and-dc-content/>